

Programmazione avanzata

MAX-HEAP

Alberi

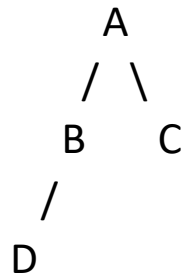
- Ogni nodo ha un *padre* (a parte la radice) e uno o più *figli* (a parte le foglie).
- Chiamiamo:
 - *nodi interni*: tutti i nodi dei cammini tra la radice e le foglie
 - *profondità* (di un nodo N): la distanza di N dalla radice
 - *altezza* (dell'albero): la distanza massima tra la radice e una foglia
 - *antenato* (di un nodo N): ogni nodo che precede N sul cammino dalla radice a N
 - *padre* (di un nodo N): il nodo che immediatamente precede N lungo il cammino dalla radice a N
 - *figlio* (di un nodo N): ogni nodo di cui N è padre
 - *fratelli* (di un nodo N): i nodi che hanno lo stesso padre di N
- Un albero è *binario* se ogni nodo ha *al più* 2 figli

Precisazioni.

In teoria dei grafi, un **albero** è un grafo che soddisfa tre proprietà fondamentali:

- Non orientato: gli archi non hanno una direzione
- Connesso: esiste un cammino tra ogni coppia di nodi. Non esistono nodi isolati.
- Aciclico: non ci sono cicli, quindi tra due nodi qualunque c'è esattamente un solo cammino.

Consideriamo un albero binario



Radice: A; foglie: C, D

- struttura: In termini di teoria dei grafi, è un grafo non orientato. Gli archi tra i nodi non hanno una direzione intrinseca e possono essere percorsi nei due sensi

- interpretazione gerarchica: scegliamo un nodo specifico, la radice, come punto di partenza. Diciamo che stiamo considerando un albero *radicato* ([https://en.wikipedia.org/wiki/Arborescence_\(graph_theory\)](https://en.wikipedia.org/wiki/Arborescence_(graph_theory))) In questo modo definiamo una direzione implicita degli archi, che vanno dalla radice verso le foglie.

Stiamo dunque sovrapponendo una informazione di tipo applicativo su un grafo di per se non orientato.. A tutti i fini pratici possiamo considerarlo come grafo orientato.

Alberi

Definizioni

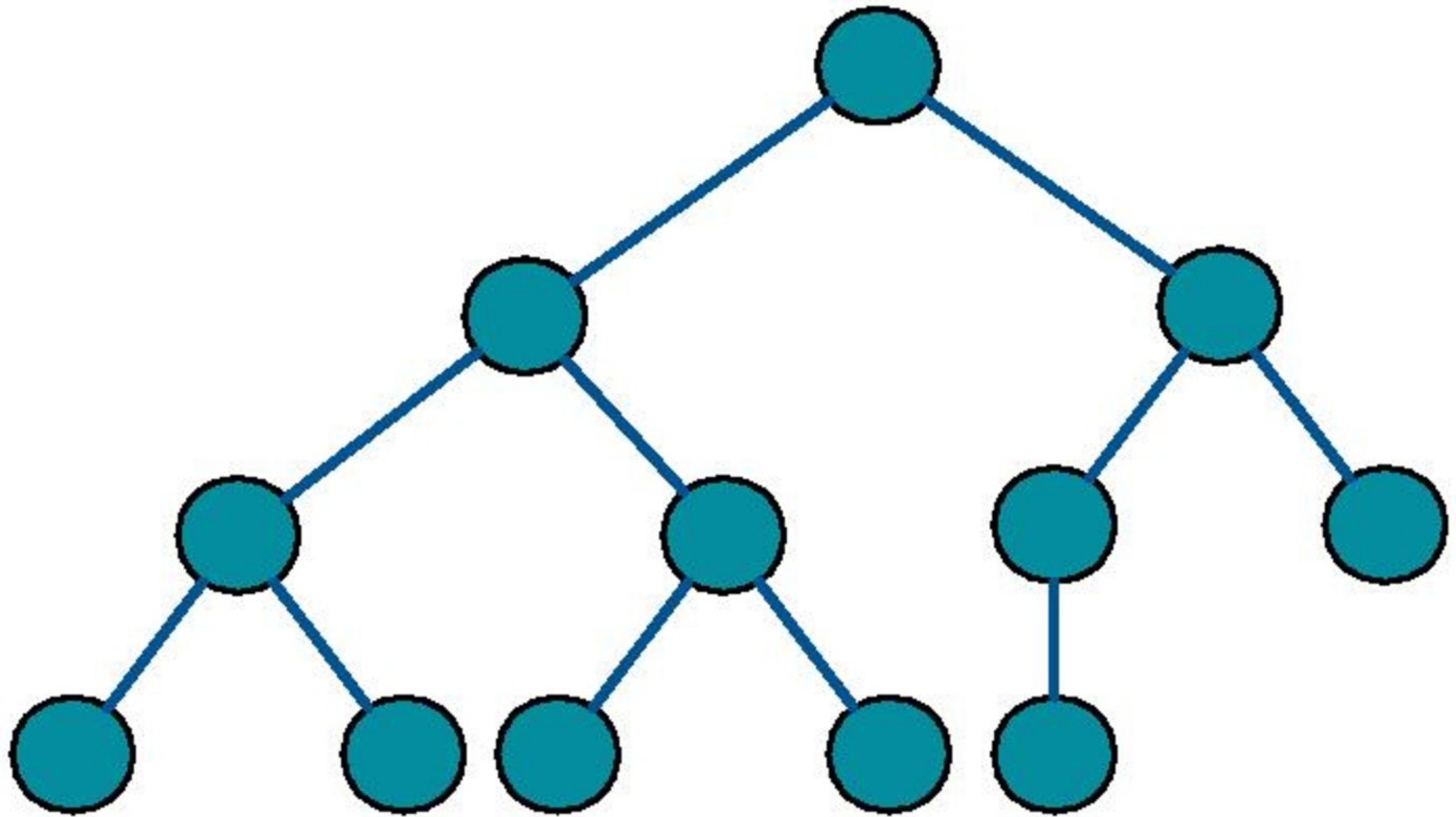
Un albero binario è un albero i cui nodi hanno **grado** compreso tra 0 e 2.

Grado: numero di archi incidenti sul nodo.

Un albero binario completo è un albero binario in cui ogni livello, tranne eventualmente l'ultimo, è completamente pieno, e tutti i nodi sono il più a sinistra possibile.

Un albero viene chiamato albero quasi completo se l'ultimo livello non è completamente pieno.

Alberi



Heap

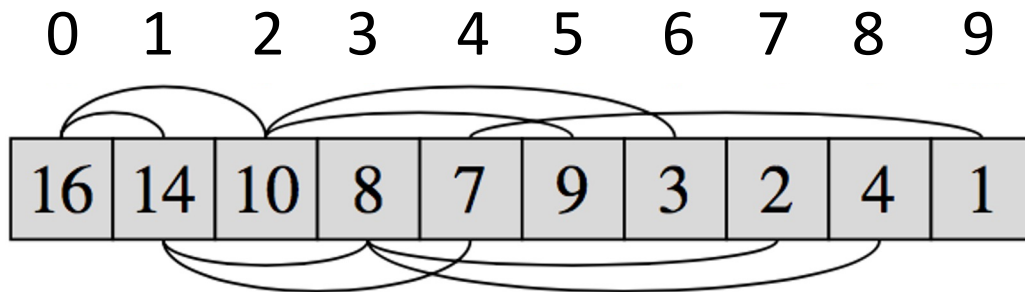
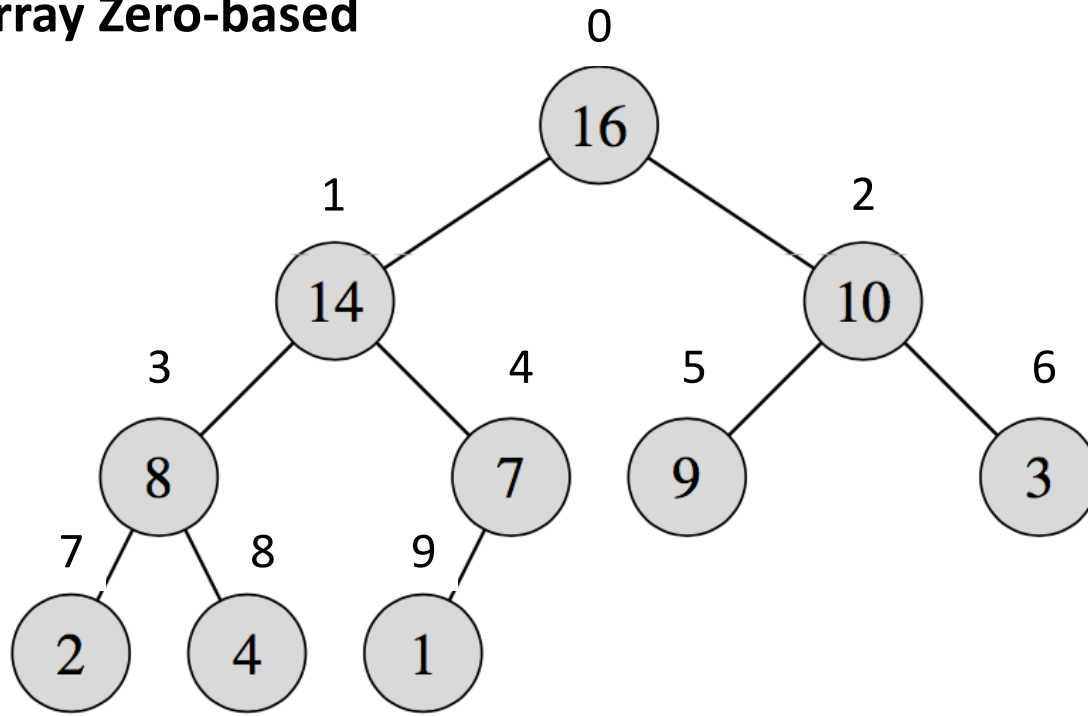
Un heap (binario) è una struttura dati composta da un array che possiamo considerare come un **albero binario quasi completo**.

Ogni nodo dell'albero corrisponde a un elemento dell'array che memorizza il valore del nodo.

Tutti i livelli dell'albero sono completamente riempiti, tranne eventualmente l'ultimo che può essere riempito da sinistra fino a un certo punto.

Heap

Array Zero-based



$$Parent(i) = \left\lfloor \frac{i-1}{2} \right\rfloor$$

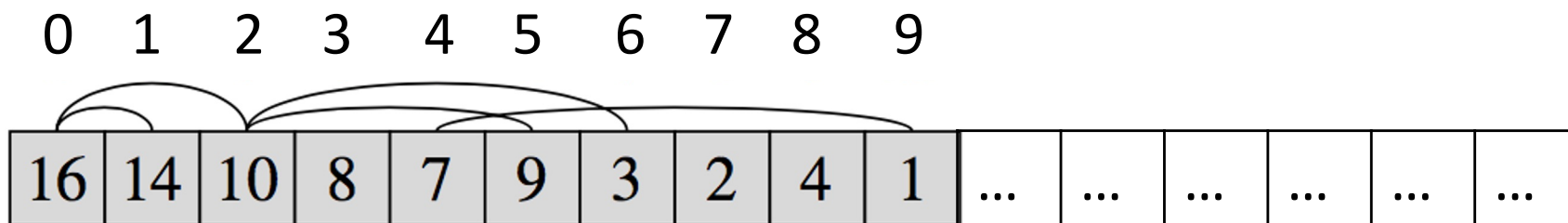
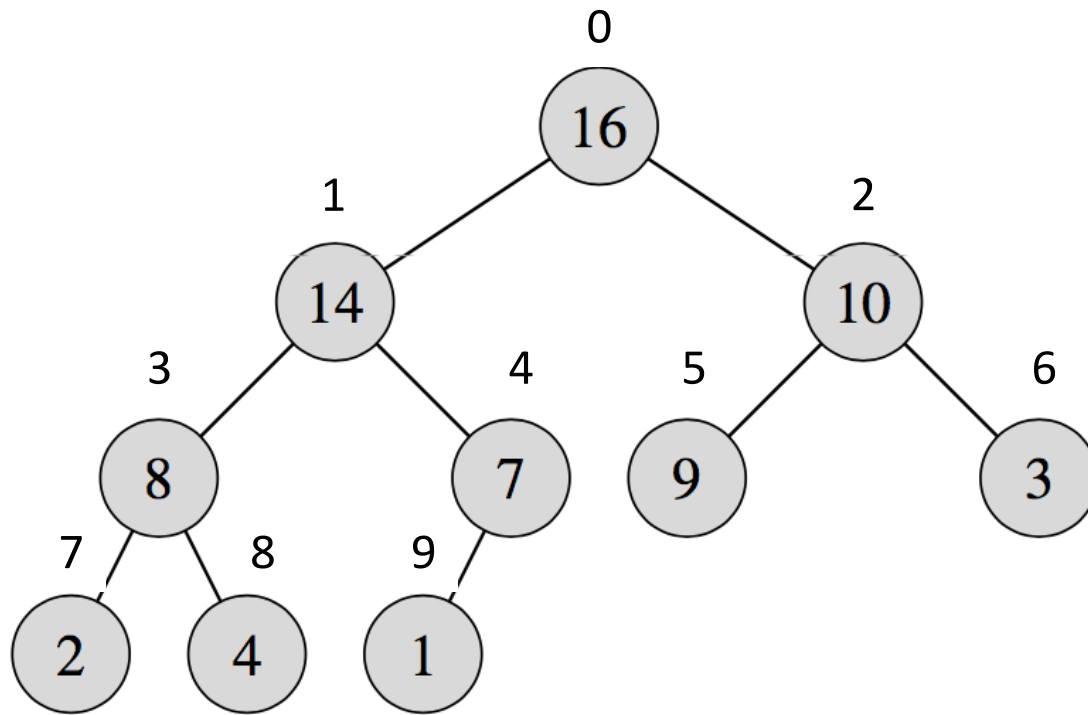
$$Left(i) = 2i + 1$$

$$Right(i) = 2(i + 1)$$

l j

intero
inferiore

Heap



heap-size



len(A)

Heap

Esercizio

1. Implementate le funzioni parent, left, right e testatele esaustivamente utilizzando il vettore nella figura precedente
2. separate il main dall'heap utilizzando un file per il main e uno o più file per l'heap.

Max Heap

Max-heap: i valori dei nodi soddisfano la seguente proprietà: per ogni nodo i diverso dalla radice, si ha

$$A[\text{PARENT}(i)] \geq A[i]$$

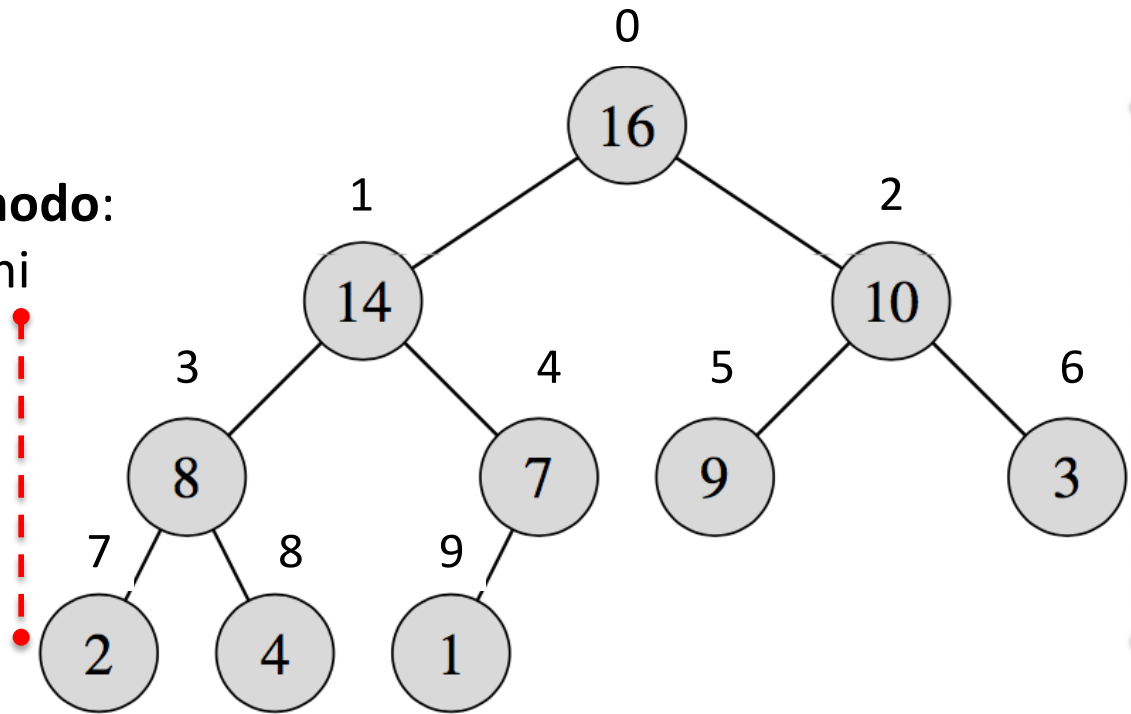
il valore di un nodo e' al massimo il valore di suo padre

- L'elemento piu' grande di un max-heap e' memorizzato nella radice
- Il sottoalbero di un nodo contiene valori minori o uguali di quello contenuto nel nodo stesso

(esiste anche min-heap, con proprietà opposte)

Heap

Altezza di un nodo:
numero di archi
fino alla foglia



Altezza di un heap:
altezza della radice

L'altezza di un heap è $\lfloor \log_2 n \rfloor$

Max Heap

Se un albero binario gode della proprietà di Max-heap, il valore massimo si trova sempre nella radice.

Sfruttando questa proprietà possiamo ordinare un vettore seguendo questi passi, illustrati nel dettaglio più avanti:

- si crea un Max-heap
 - si prende l'elemento in radice, eliminandolo dall'albero. Otteniamo l'elemento più grande.
 - si ripristina la proprietà di max-heap
 - si prende il nuovo elemento in radice, eliminandolo dall'albero. Otteniamo il secondo elemento più grande.
- e così via.

funzioni

- MAX-HEAPIFY
- BUILD-MAX-HEAP
- HEAP SORT

MAX-HEAPIFY

MAX-HEAPIFY

Funzione che permette di ripristinare le proprietà dell'heap **nel caso in cui un nodo sia più piccolo dei suoi figli.**

Quando si può applicare?

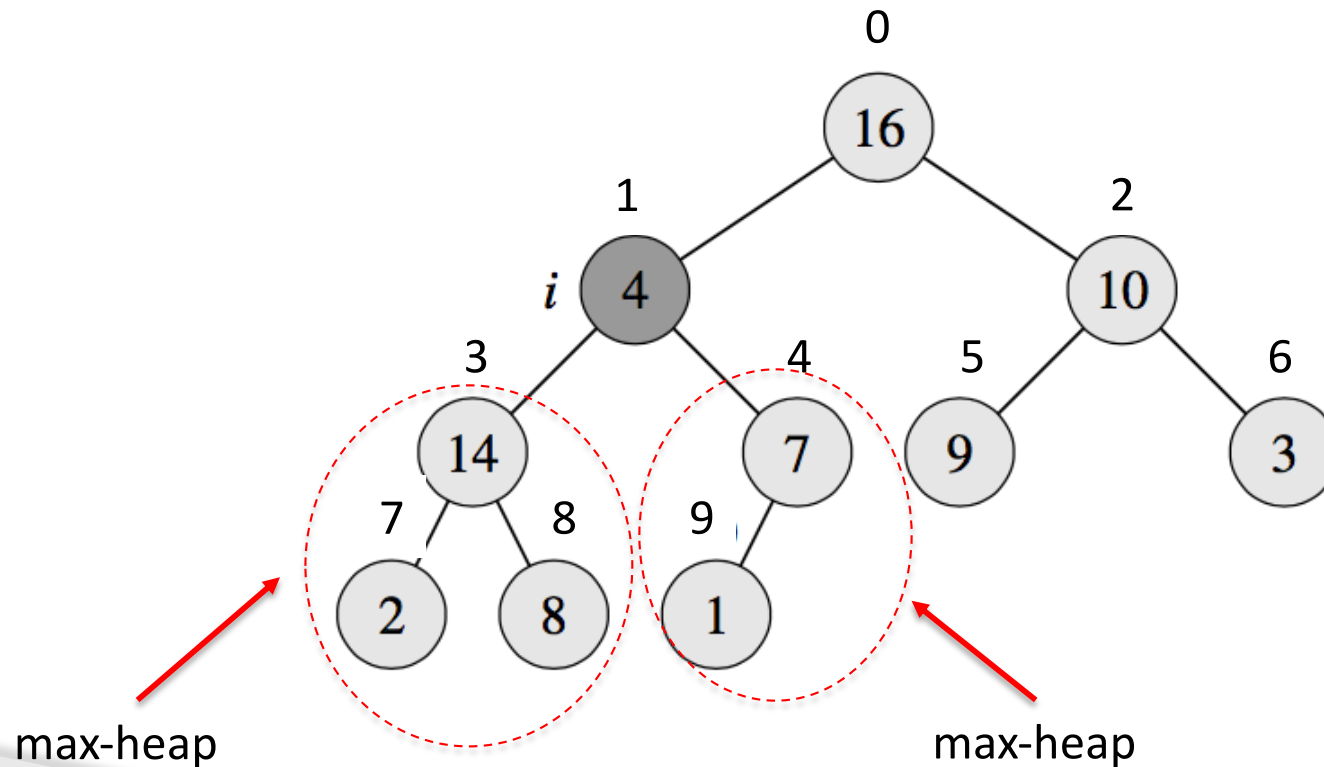
Partiamo da un MAX-HEAP e decrementiamo il valore di un nodo. Il decremento **POTREBBE** violare la proprietà di MAX-HEAP.

MAX-HEAPIFY riordina l'albero in modo da ripristinare la proprietà.

MAX-HEAPIFY

MAX-HEAPIFY

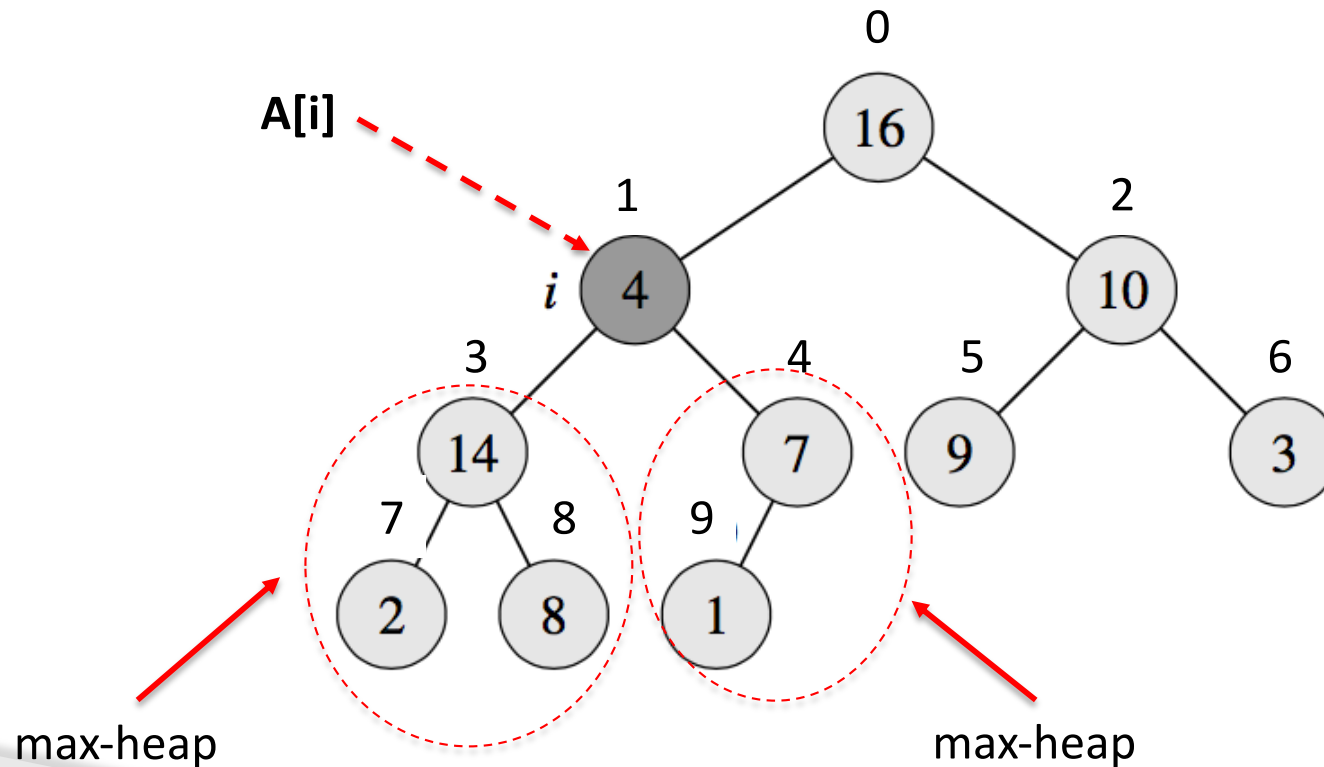
Funzione che permette di ripristinare le proprietà dell'heap nel caso in cui un nodo sia più piccolo dei suoi figli.



MAX-HEAPIFY

MAX-HEAPIFY

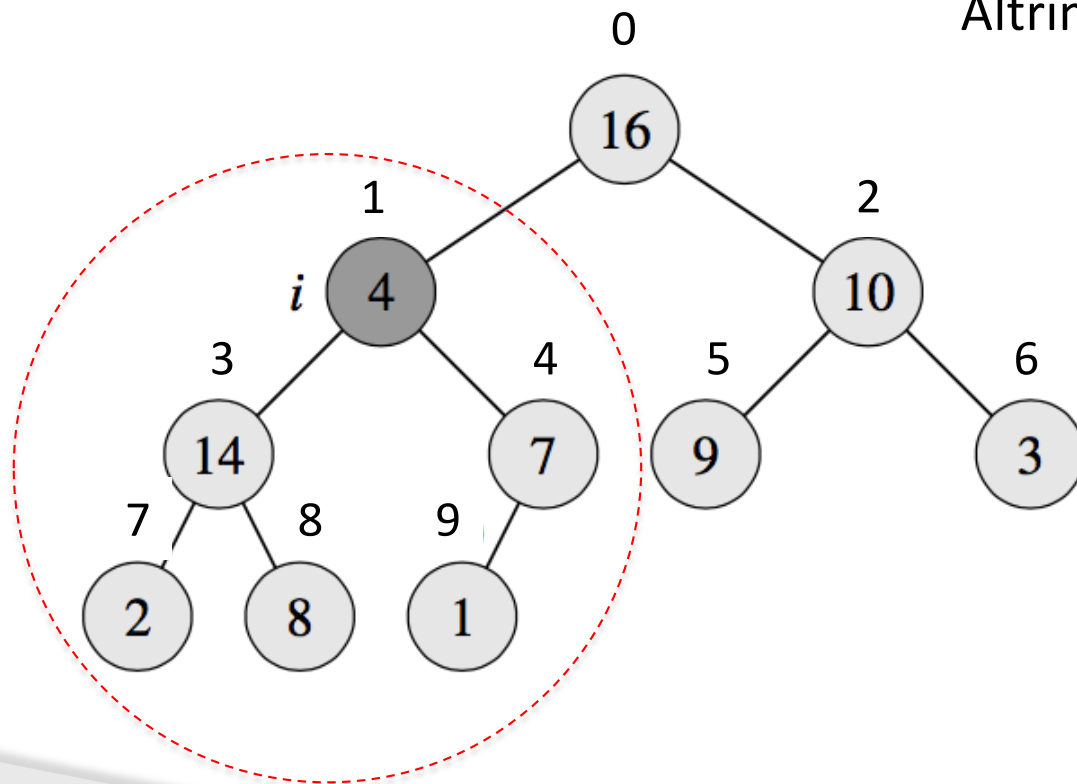
Gli alberi binari con radici in $\text{LEFT}(i)$ e $\text{RIGHT}(i)$ sono max-heap, ma $A[i]$ può essere più piccolo dei suoi figli, violando così la proprietà del max-heap.



MAX-HEAPIFY

MAX-HEAPIFY fa scendere $A[i]$ nel max-heap in modo che il sottoalbero con radice di indice i diventi un max-heap.

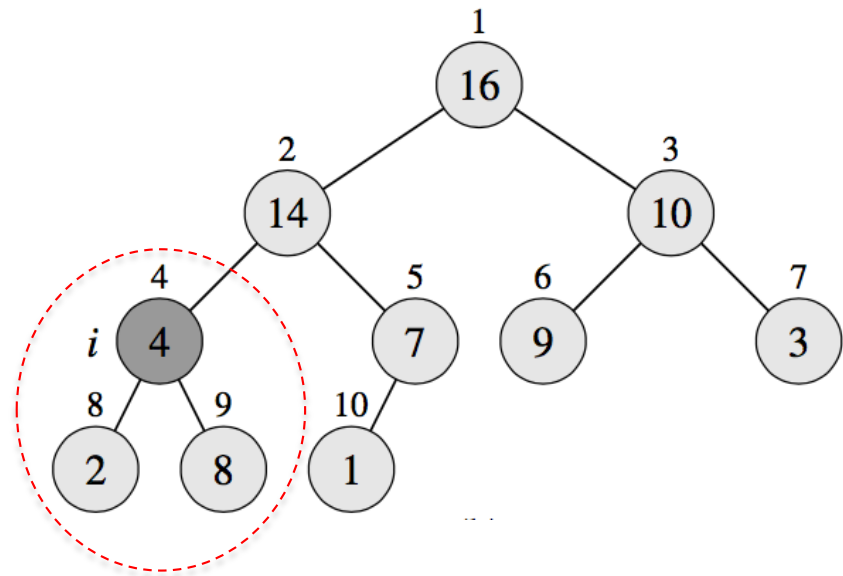
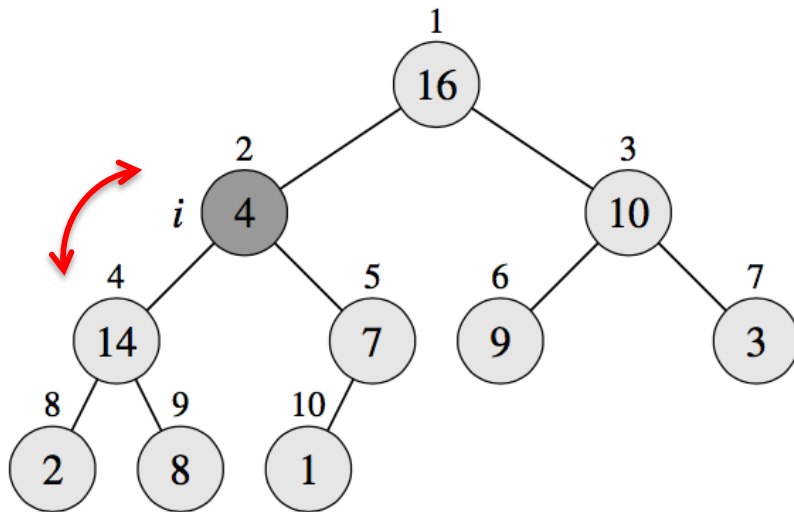
Attenzione: devo scambiare il 4 con il 14 e non con il 7
Altrimenti non ottengo un max-heap!



MAX-HEAPIFY

MAX-HEAPIFY

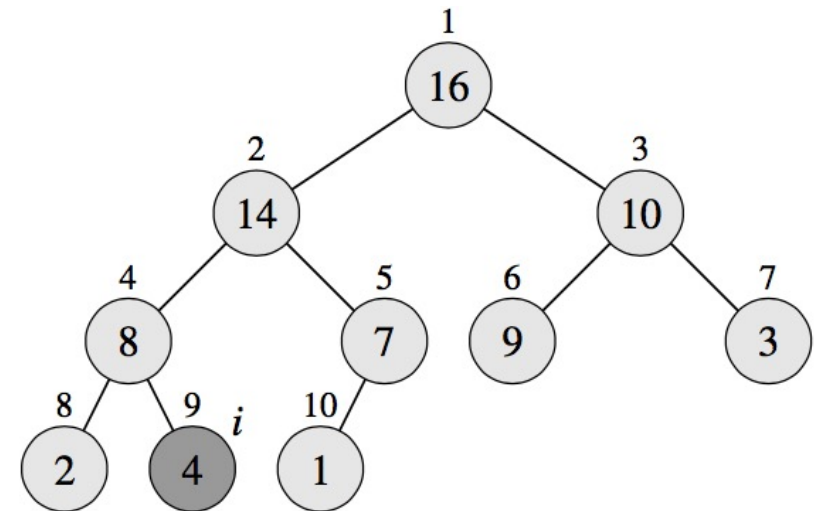
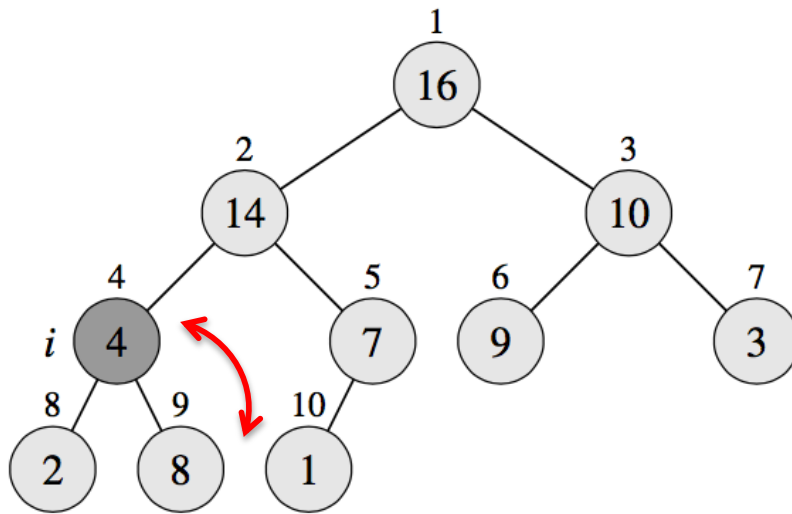
- $\text{max} = \max(A[\text{left}(i)], A[\text{right}(i)])$
- se $A[i] > \text{max}$, termina
- altrimenti scambia $A[i]$ col massimo



MAX-HEAPIFY

MAX-HEAPIFY

- $\text{max} = \max(A[\text{left}(i)], A[\text{right}(i)])$
- se $A[i] > \text{max}$, termina
- altrimenti scambia $A[i]$ col massimo
- applica ricorsivamente



MAX-HEAPIFY

Esercizio

Implementate MAX-HEAPIFY in modo ricorsivo

Possibile implementazione:

```
max_heapify(int* A, int i, int heap_size)
```

MAX-HEAPIFY

```
max_heapify(A,i,heap_size)
Calcolo l, r // figli di i
se l è valido,
    se A[l]>A[i], MASSIMO=A[l] .
    Altrimenti MASSIMO=A[i]
se r è valido,
    se A[r]>MASSIMO, MASSIMO=A[r] .
    Altrimenti PASS;
se MASSIMO!=A[i],
    scambia
    max_heapify(A,idx_max,heap_size)
altrimenti STOP
```

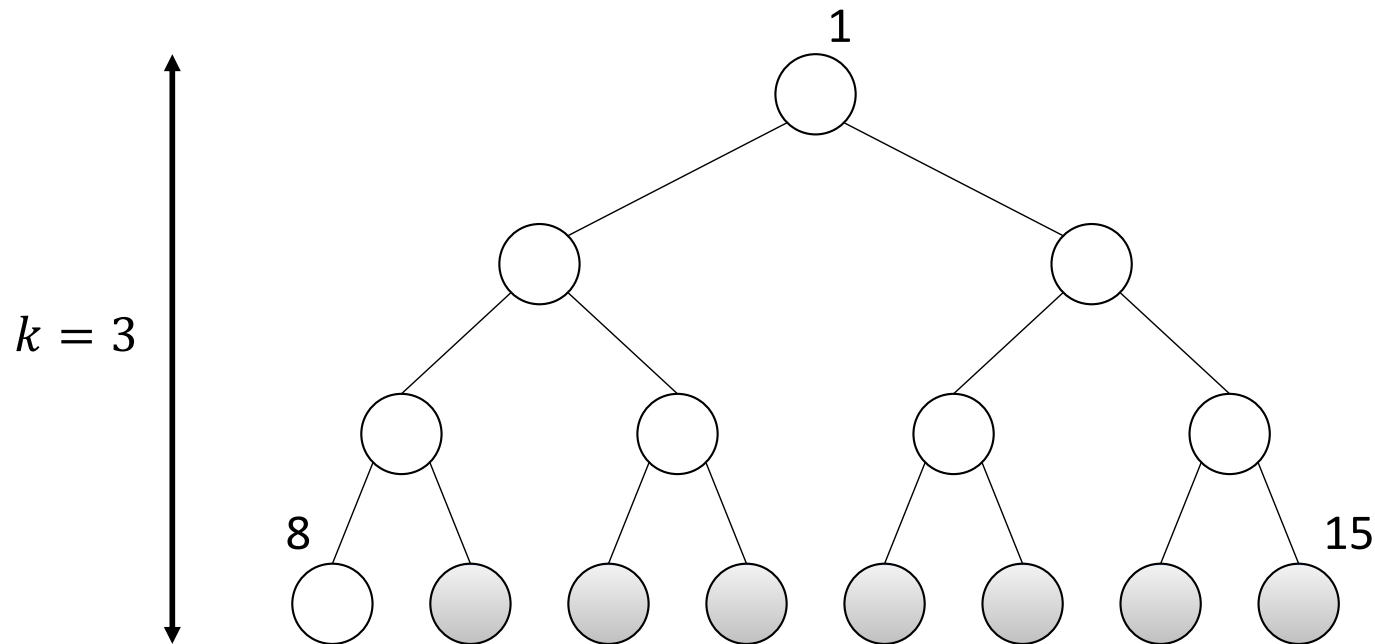
MAX-HEAPIFY

max_heapify: calcolo complessità computazionale

Premessa:

Un albero binario (completo o quasi completo) di **altezza k** ha numero di nodi n compreso tra 2^k e $2^{k+1} - 1$

Esempio: $k = 3 \rightarrow n$ compreso tra $2^3 = 8$ e $2^4 - 1 = 15$



MAX-HEAPIFY

Infatti il **massimo numero di nodi** (albero completo) è:

$n_{max}(k = 0) = 2^0$ (altezza 0, ho solo un nodo)

$n_{max}(k = 1) = 2^0 + 2^1$ (altezza 1, ho 2 nodi + livello precedente)

...

$$n_{max}(k) = 2^0 + 2^1 + \dots + 2^k = \sum_{i=0}^k 2^i = 2^{k+1} - 1$$

è una serie geometrica di ragione 2

$$\sum_{i=0}^k r^i = \frac{r^{k+1} - 1}{r - 1}$$

$$\sum_{i=0}^k 2^i = \frac{2^{k+1} - 1}{2 - 1} = 2^{k+1} - 1$$

MAX-HEAPIFY

Il **minimo numero di nodi** è pari al numero di nodi di un albero di altezza $k - 1$ aumentato di 1

$$n_{min}(k) = n_{max}(k - 1) + 1 = 2^k$$

MAX-HEAPIFY

Calcolo complessità computazionale

max_heapify agisce sul nodo i e poi, ricorsivamente, su **uno** dei sottoalberi sinistro o destro.

- $\text{max} = \max(A[\text{left}(i)], A[\text{right}(i)])$
- se $A[i] > \text{max}$, termina
- altrimenti scambia $A[i]$ col massimo
- applica ricorsivamente

I primi tre step hanno un costo $T \leq T_{\text{costante}}$ (es: costo ≤ 3)

Nel caso peggiore devo applicare max-heapify dalla radice fino all'ultima foglia. Ad ogni iterazione 'scendo' di un livello nell'albero. Quindi nel caso peggiore ho un costo $T_{\text{costante}} \cdot \log_2(n)$, essendo $\log_2(n)$ il numero di livelli dell'albero.

Complessità computazionale: $T(n) = O(\lg_2 n)$

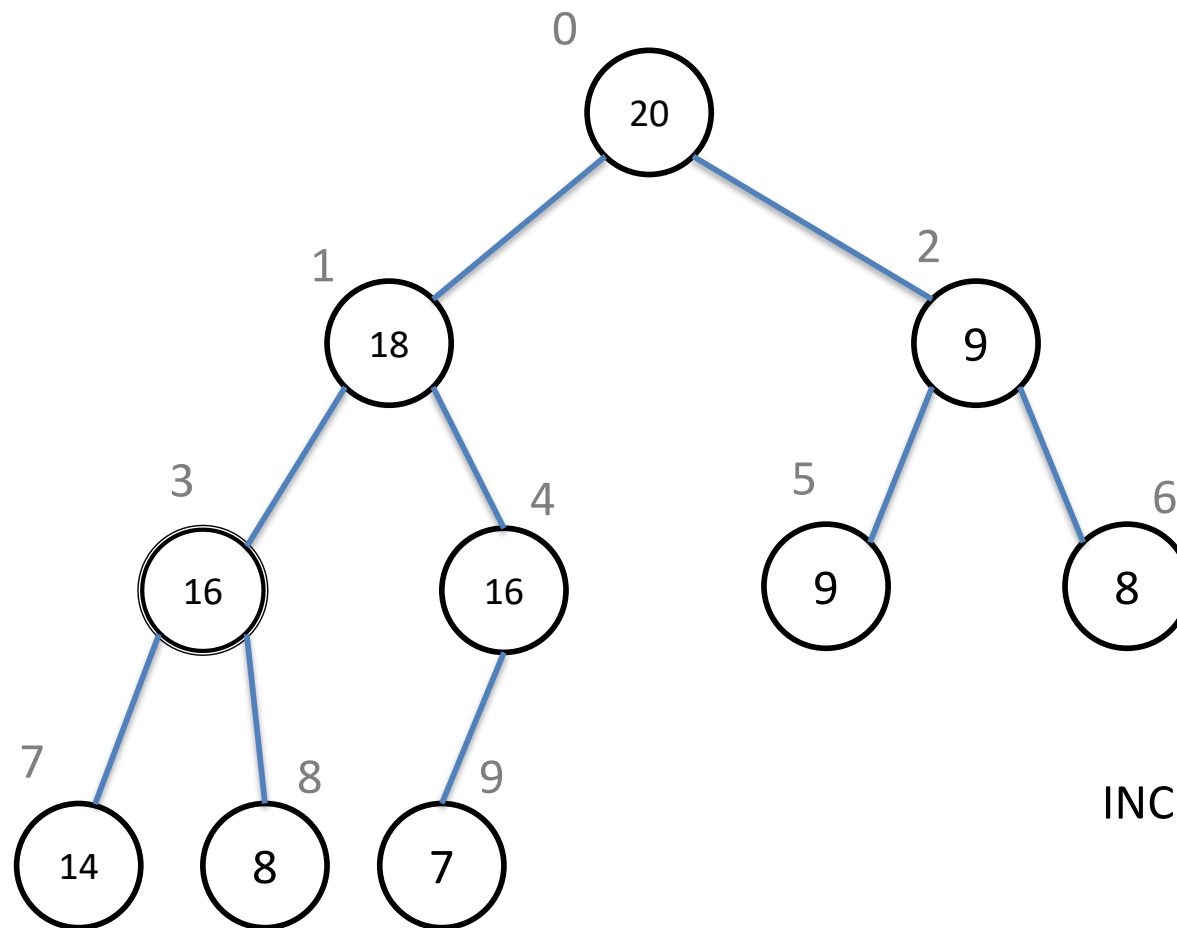
MAX-HEAPIFY

Nota su MAX-HEAPIFY

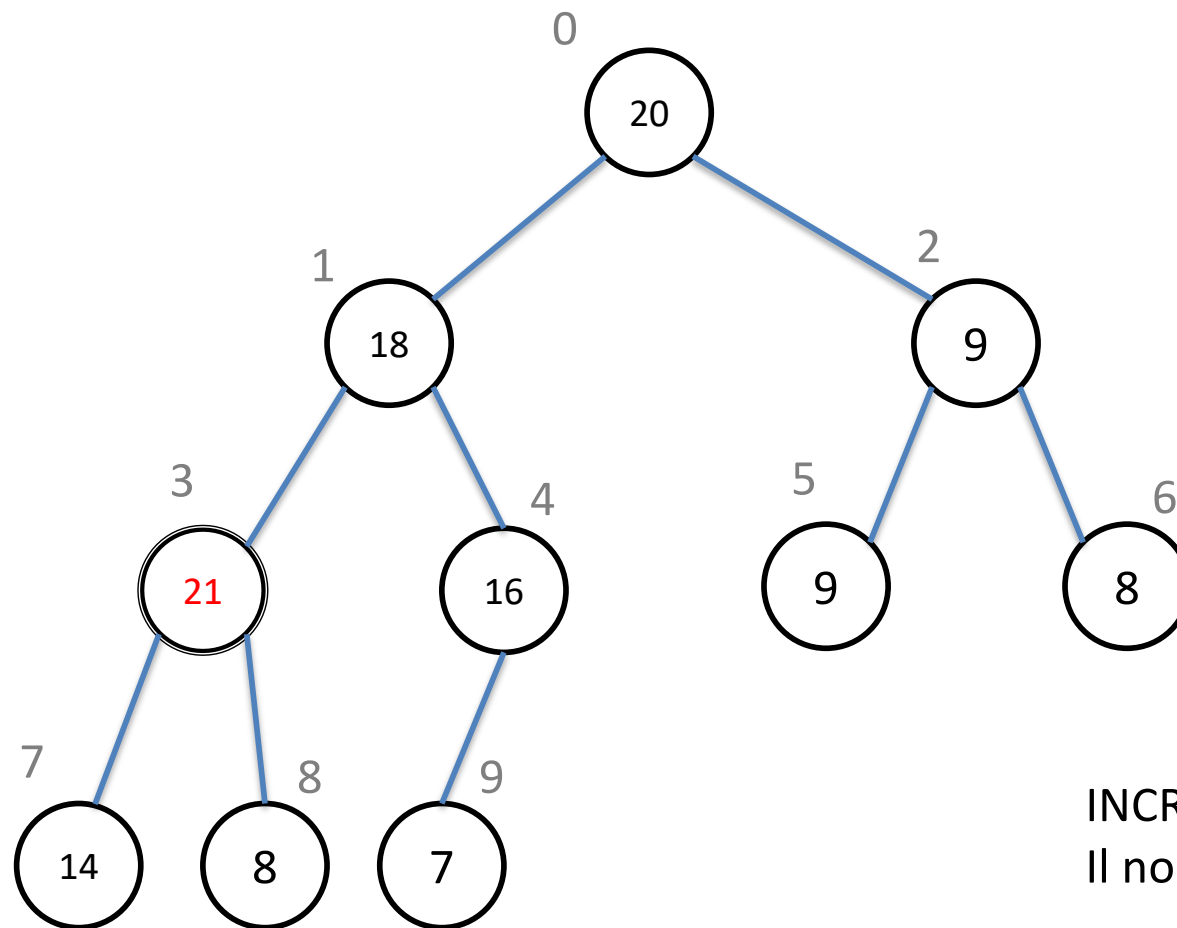
Funzione che permette di ripristinare le proprietà dell'heap **nel caso in cui un nodo sia più piccolo dei suoi figli.**

Più precisamente, partendo da un MAX-HEAP, se decremento il valore del nodo i in modo da violare la proprietà di max-heap, la posso ripristinare applicando max-heapify al nodo i

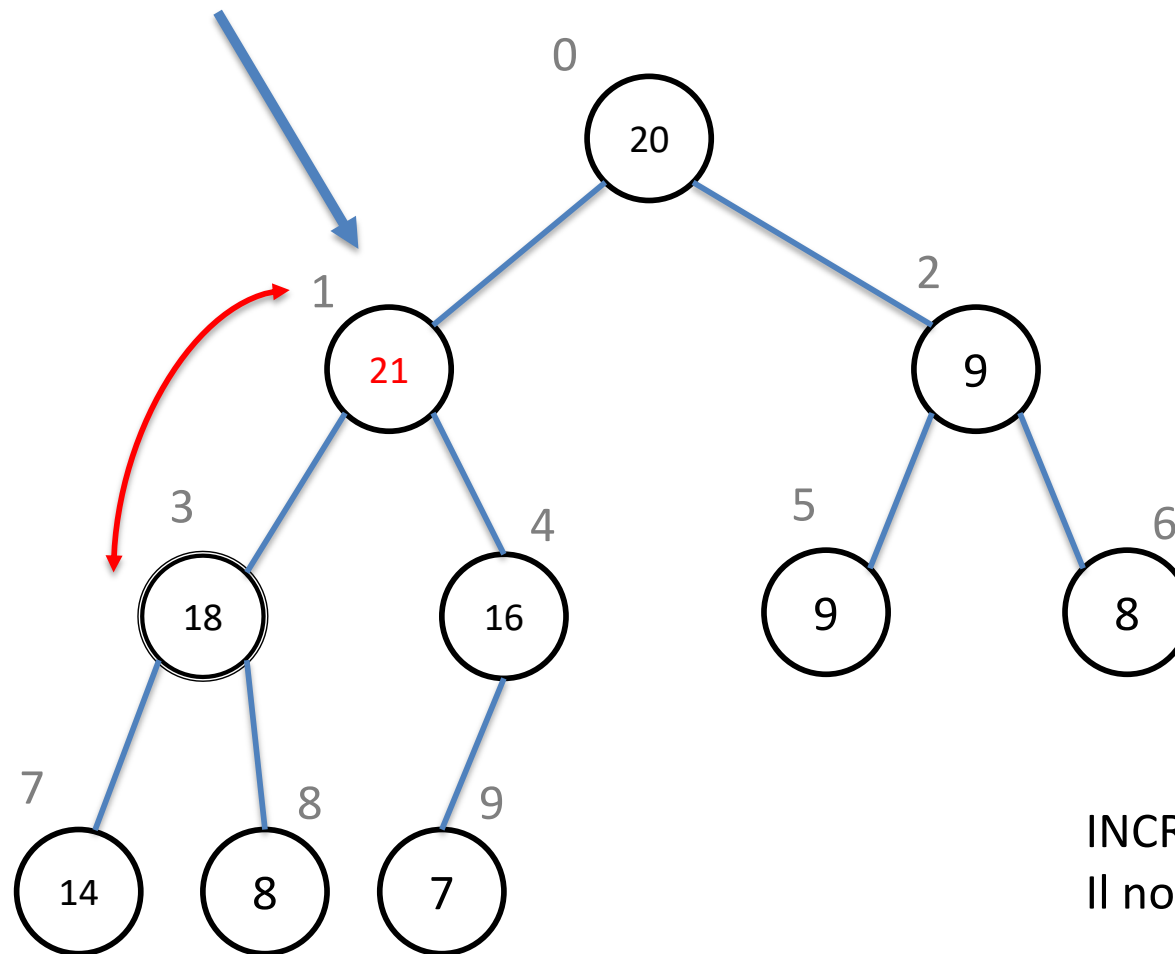
Cosa succede se invece AUMENTO il valore del nodo i ? posso ripristinare la proprietà di max-heap applicando max-heapify a $\text{parent}(i)$?



INCREMENTO IL NODO 3



INCREMENTO IL NODO 3
Il nodo 1 è maggiore dei suoi figli



INCREMENTO IL NODO 3
Il nodo 1 è maggiore dei suoi figli

Applicare max-heapify al nodo 1
non ripristina la proprietà.

BUILD-MAX-HEAP

BUILD-MAX-HEAP

Funzione che permette di costruire un max-heap

Convertiamo un array di n elementi in un max-heap di n elementi

A	4	1	3	2	16	9	10	14	8	7
-----	---	---	---	---	----	---	----	----	---	---

BUILD-MAX-HEAP

BUILD-MAX-HEAP

Funzione che permette di costruire un max-heap

Convertiamo un array di n elementi in un max-heap di n elementi

A	4	1	3	2	16	9	10	14	8	7
---	---	---	---	---	----	---	----	----	---	---

È banale applicare insertion-sort o simili. Ma avrei $O(n^2)$.

Con merge sort avrei $O(n \log n)$

Invece **so che max-heapify è $O(\lg n)$. Provo a sfruttare questa proprietà**

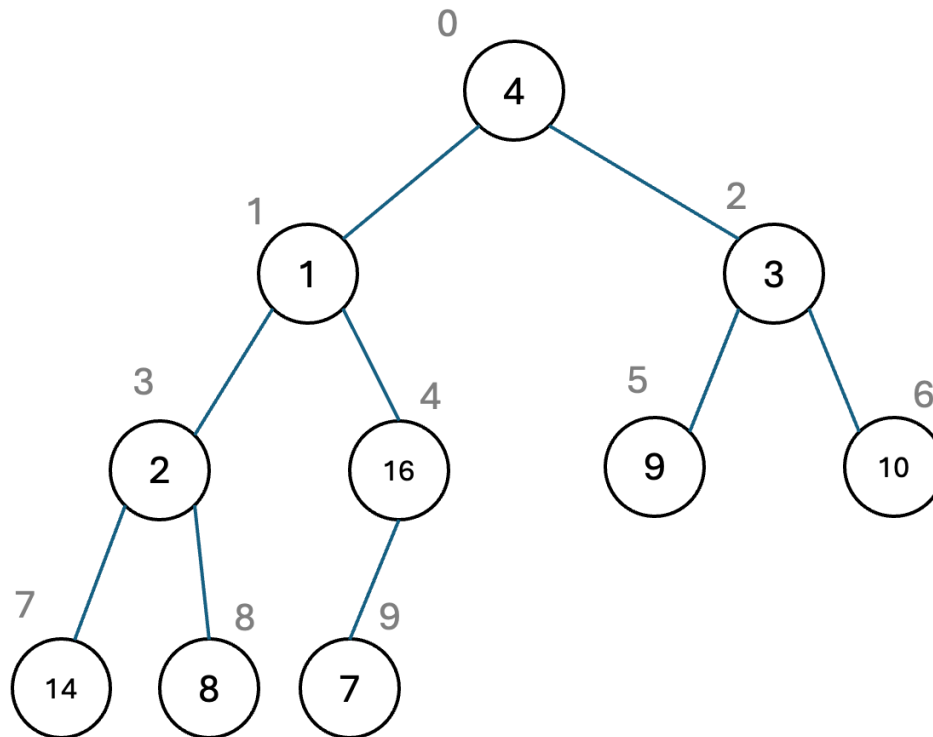
BUILD-MAX-HEAP

BUILD-MAX-HEAP

Dove posso applicare max_heapify()?

A	4	1	3	2	16	9	10	14	8	7
---	---	---	---	---	----	---	----	----	---	---

n=10



$$\text{Parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor$$

$$\text{Left}(i) = 2i + 1$$

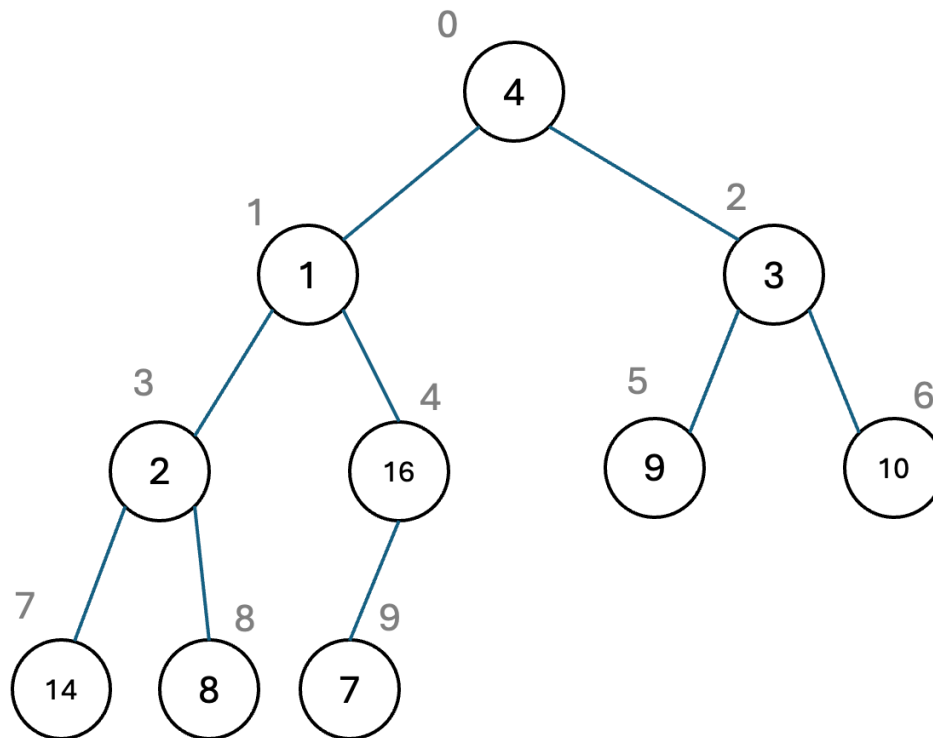
$$\text{Right}(i) = 2(i + 1)$$

BUILD-MAX-HEAP

BUILD-MAX-HEAP

Dove posso applicare `max_heapify()`?

A	4	1	3	2	16	9	10	14	8	7
---	---	---	---	---	----	---	----	----	---	---



- Quali sottoalberi sono sicuramente max-heap?

- Qual è il sottoalbero più piccolo che *potrebbe* non essere max-heap?

BUILD-MAX-HEAP

Gli indici dei nodi variano tra 0 e $(n-1)$

I **nodi interni** (non foglia) hanno almeno un figlio sinistro

$$\text{left}(i) = 2i+1$$

$$\text{Deve essere } \text{left}(i) < n \quad \rightarrow \quad 2i+1 < n \quad \rightarrow \quad i < (n-1)/2$$

$$\mathbf{n \text{ pari}} \rightarrow i \in \{0, 1, 2, \dots, \frac{n}{2} - 1\}$$

$(n-1)/2 = n/2 - 1/2$ non è intero; l'intero precedente è $n/2 - 1$

$$\mathbf{n \text{ dispari}} \rightarrow i \in \{0, 1, 2, \dots, \left\lfloor \frac{n}{2} \right\rfloor - 1\}$$

$(n-1)/2$ è intero e vale l'uguaglianza $\frac{n-1}{2} = \left\lfloor \frac{n}{2} \right\rfloor$

in entrambi i casi i nodi interni sono

$$\{0, 1, 2, \dots, \left\lfloor \frac{n}{2} \right\rfloor - 1\}$$

BUILD-MAX-HEAP

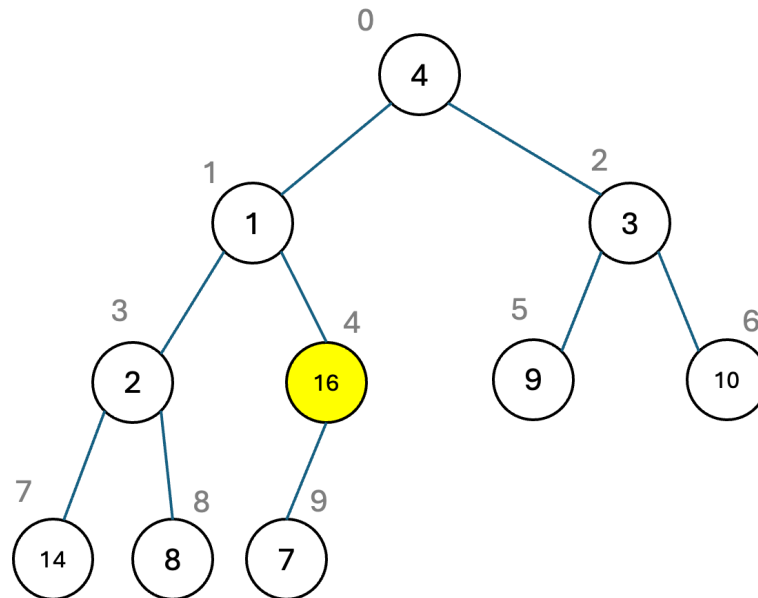
BUILD-MAX-HEAP

parto da $i = \left\lfloor \frac{n}{2} \right\rfloor - 1$ e applico **max_heapify(A,i)**

decremento i e applico **max_heapify(A,i)**

...

A	4	1	3	2	16	9	10	14	8	7
---	---	---	---	---	----	---	----	----	---	---



BUILD-MAX-HEAP

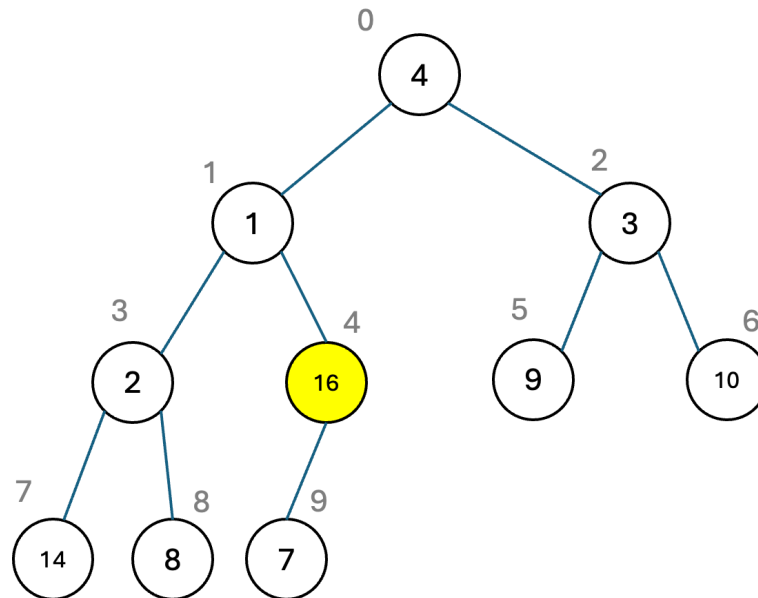
BUILD-MAX-HEAP

parto da $i = \left\lfloor \frac{n}{2} \right\rfloor - 1$ e applico **max_heapify(A,i)**

decremento i e applico **max_heapify(A,i)**

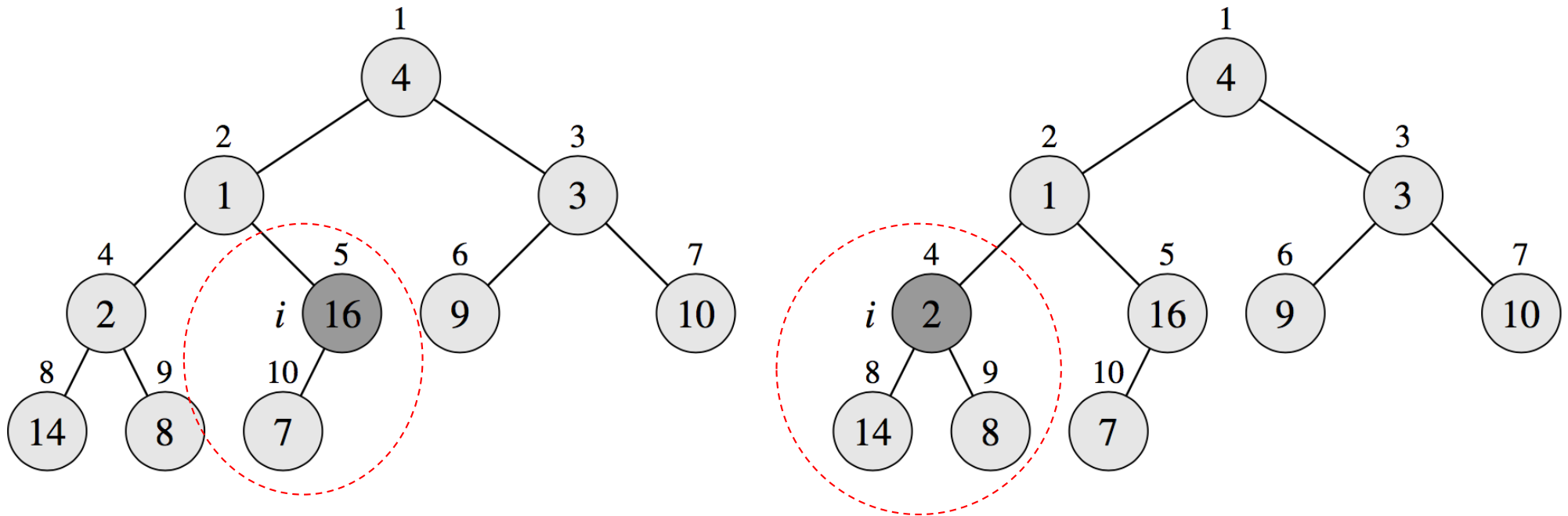
...

A	4	1	3	2	16	9	10	14	8	7
---	---	---	---	---	----	---	----	----	---	---



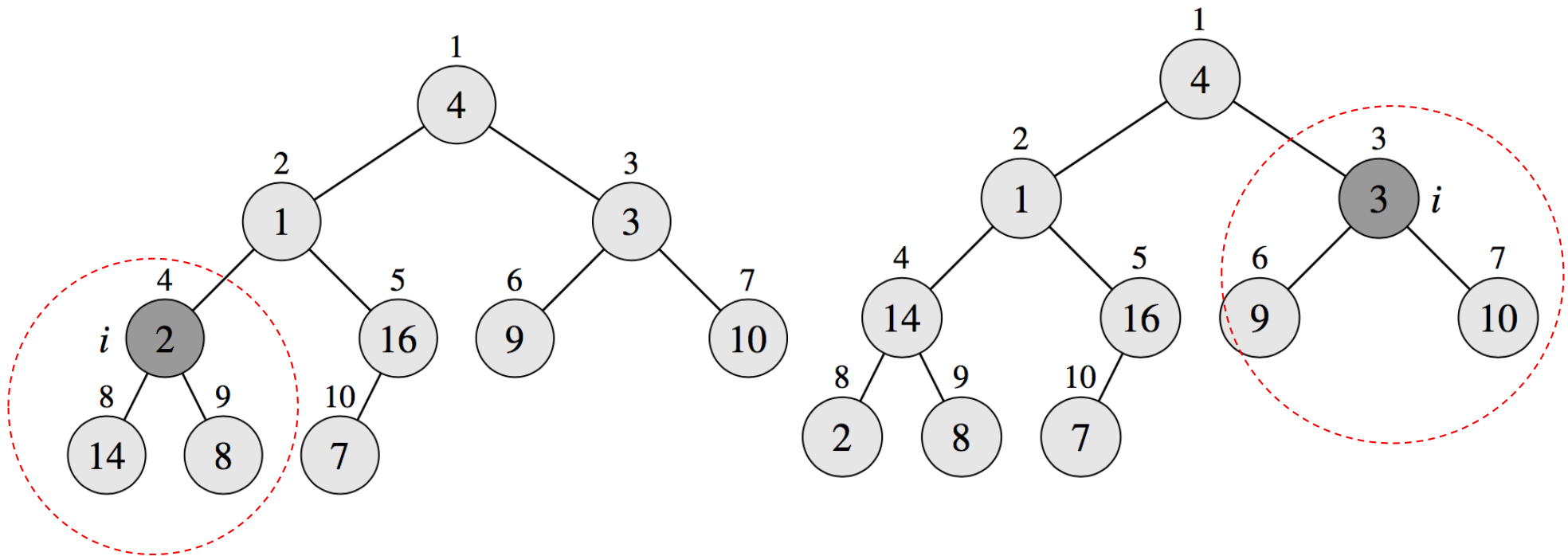
BUILD-MAX-HEAP

...



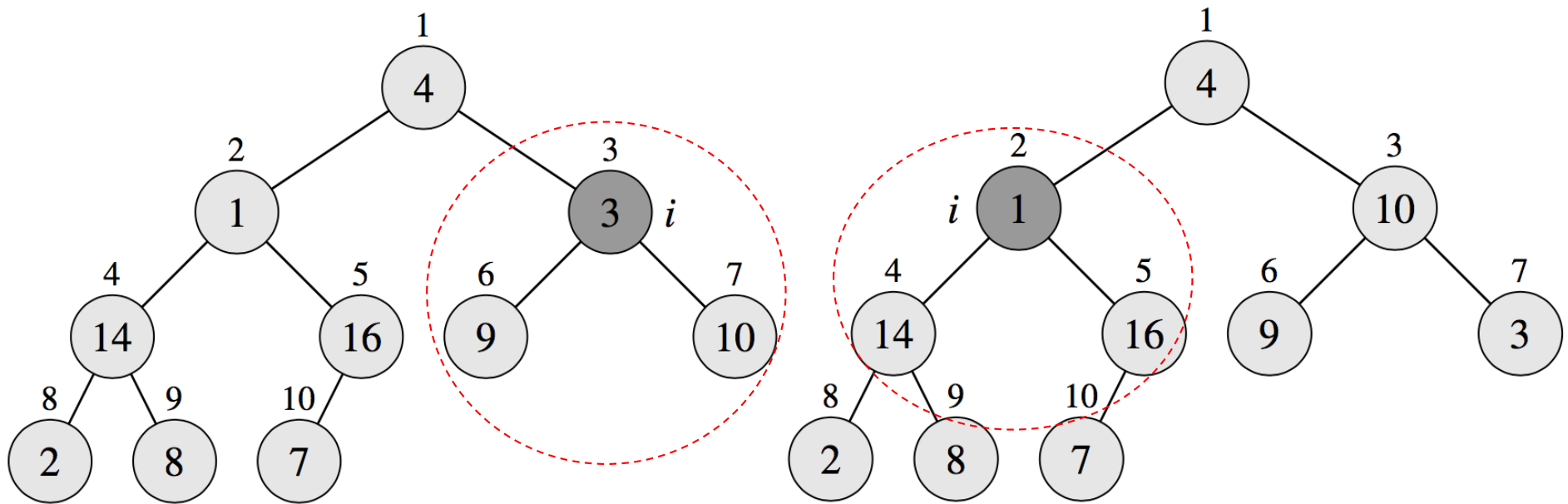
Attenzione qui gli indici partono da 1!
slides da aggiornare

BUILD-MAX-HEAP

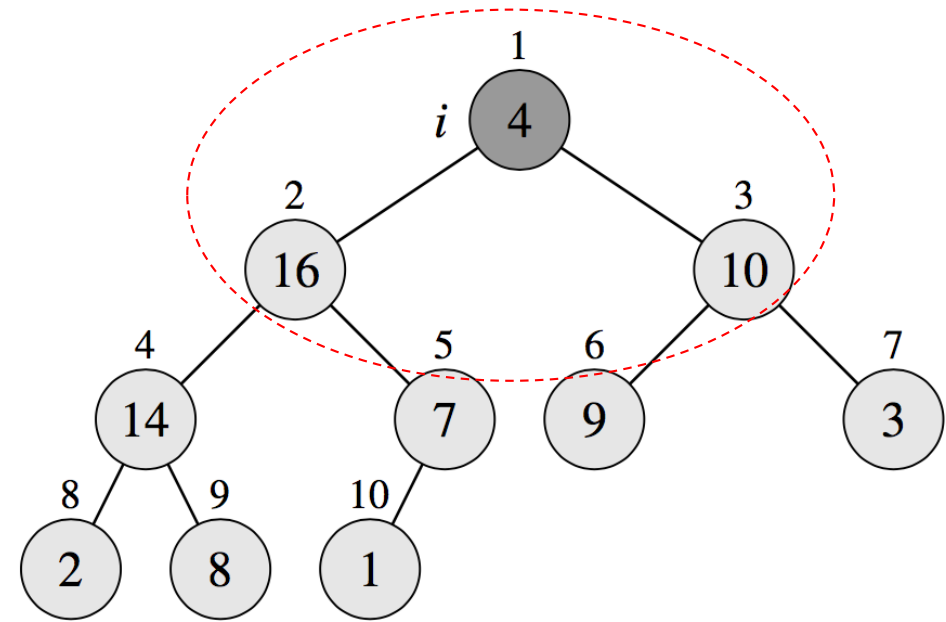
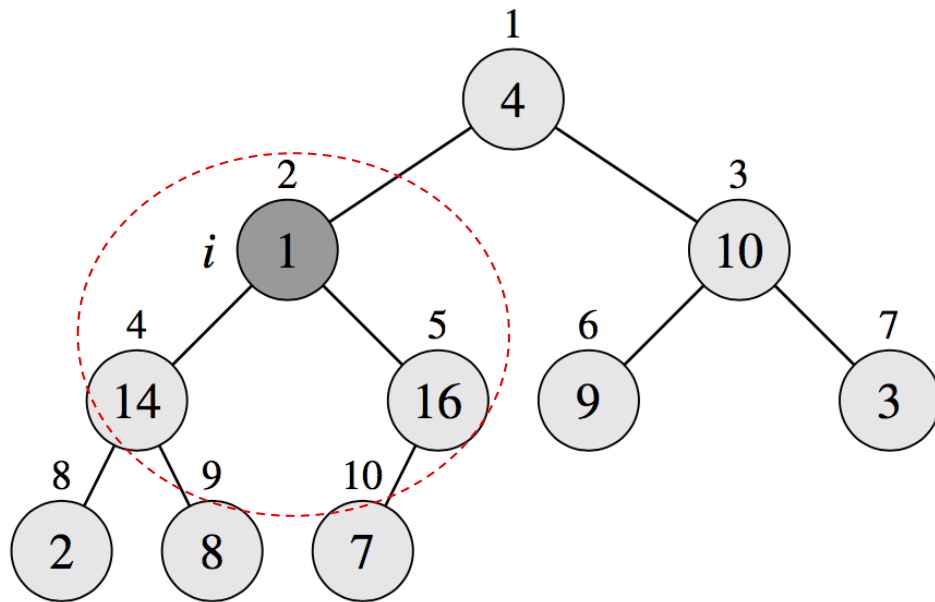


BUILD-MAX-HEAP

Ogni volta che viene chiamata la procedura MAX-HEAPIFY per un nodo, i **due sottoalberi del nodo sono già entrambi max-heap**.

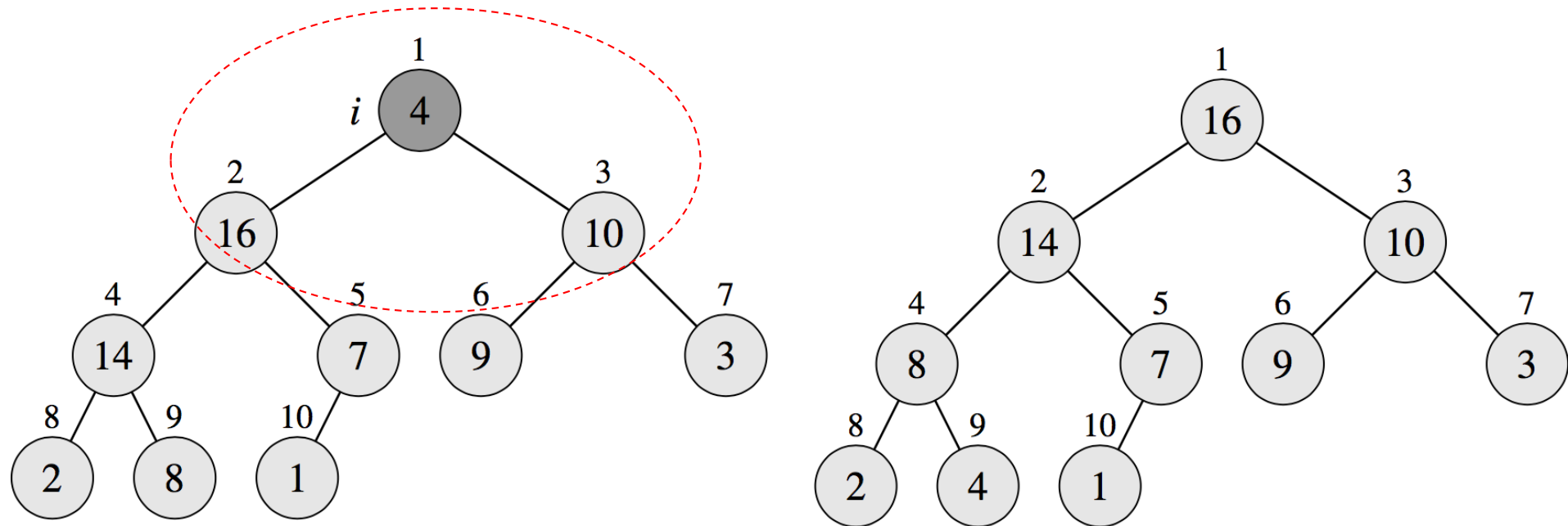


BUILD-MAX-HEAP



BUILD-MAX-HEAP

Ora ho un MAX-HEAP



BUILD-MAX-HEAP

IMPLEMENTARE IN C...

BUILD-MAX-HEAP

```
void build_max_heap(int* A, int heap_size) {  
    for (int i=(heap_size/2)-1; i>=0; i--) {  
        max_heapify(A, i, heap_size);  
    }  
}
```

BUILD-MAX-HEAP

Complessità computazionale di BUILD_MAX_HEAP

Ogni chiamata di MAX-HEAPIFY costa un tempo $O(\lg n')$ con $n' \leq n$, e ci sono $O(n)$ di queste chiamate. Quindi, il tempo di esecuzione **massimo** è limitato da $O(n \lg n)$.

Questo è un limite superiore molto ampio.

Se tutti i nodi avessero sotto di se $(\lg n)$ livelli, la complessità sarebbe $O(n \lg n)$. Ma ogni nodo ha sotto di se un numero di livelli diverso: $\lg n'$, dove n' è il numero dei nodi del sottoalbero in cui il nodo i -esimo è radice.

BUILD-MAX-HEAP

Complessità computazionale di BUILD_MAX_HEAP

La complessità computazionale sarà

$$\lg n^{(1)} + \lg n^{(2)} + \dots + \lg n^{(n)}$$

Il tempo di esecuzione varia con l'altezza del nodo ($\lg n^{(i)}$), e la maggior parte dei nodi ha altezza piccola. Pochi nodi sono vicini alla radice, e molti vicino alla base.

Si dimostra che la complessità computazionale è limitata da $O(n)$.

$$\lg n^{(1)} + \lg n^{(n)} + \dots + \lg n^{(n)} = O(n)$$

Possiamo costruire un max-heap da un array non ordinato in tempo lineare.

Heap Sort

Algoritmo HEAPSORT

Heapsort ordina un array

- costruendo prima un max-heap mediante l'algoritmo BUILD-MAX-HEAP,
- estraendo il primo elemento (che per la proprietà degli max_heap è nella radice) e scambiandolo con l'ultimo
- ripristinando la proprietà di max_heap
- e iterando il procedimento.

Heap Sort

Algoritmo HEAPSORT

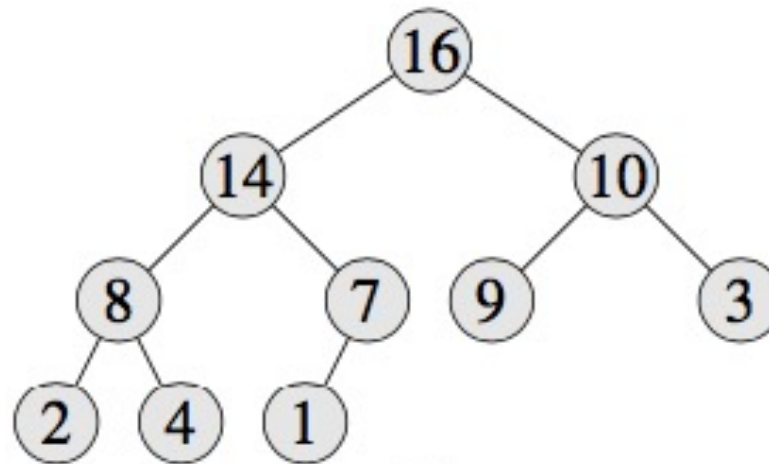
- L'elemento più grande è memorizzato nella radice (elemento $A[0]$ del vettore). Può essere inserito nella sua posizione finale corretta scambiandolo con $A[n-1]$, ed eliminandolo dall'heap.
- I figli della radice sono max-heap, ma la radice è errata. Mediante **max_heapify** ($A, 0$) ripristiniamo la proprietà max_heap.
- Riapplico al max-heap di dimensione $n-1$
- Alla fine il vettore sarà ordinato in ordine inverso e memorizzato nello stesso array che prima memorizzava l'heap-

Heap Sort

<i>A</i>	4	1	3	2	16	9	10	14	8	7
----------	---	---	---	---	----	---	----	----	---	---

Vettore DISORDINATO.

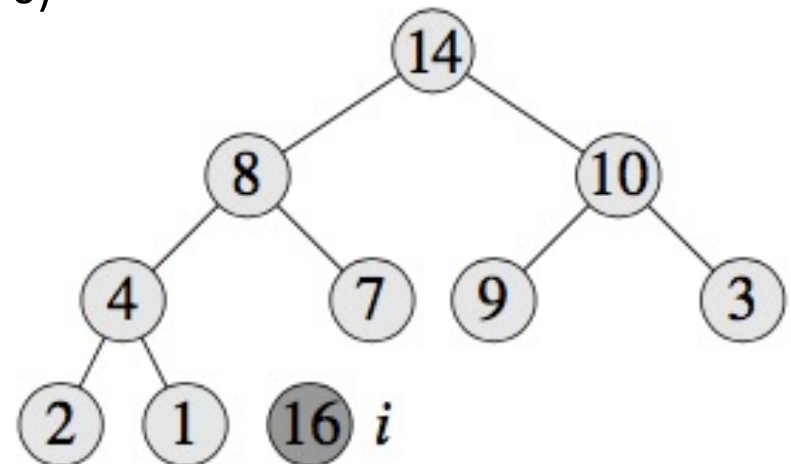
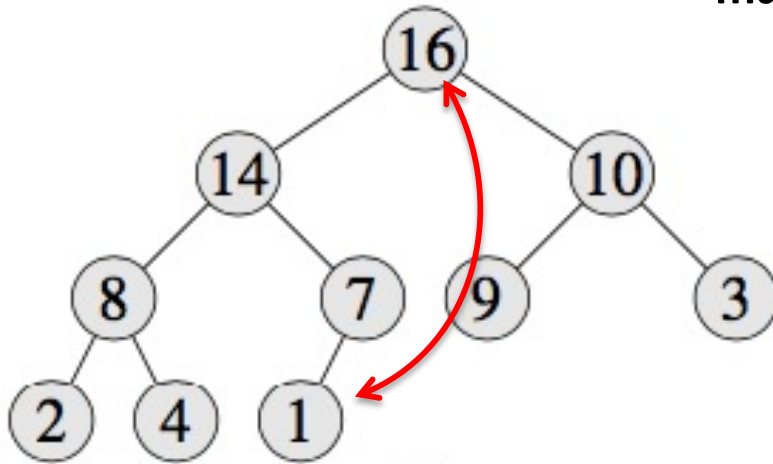
Applico BUILD_MAX_HEAP e ottengo



16	14	10	8	7	9	3	2	4	1
----	----	----	---	---	---	---	---	---	---

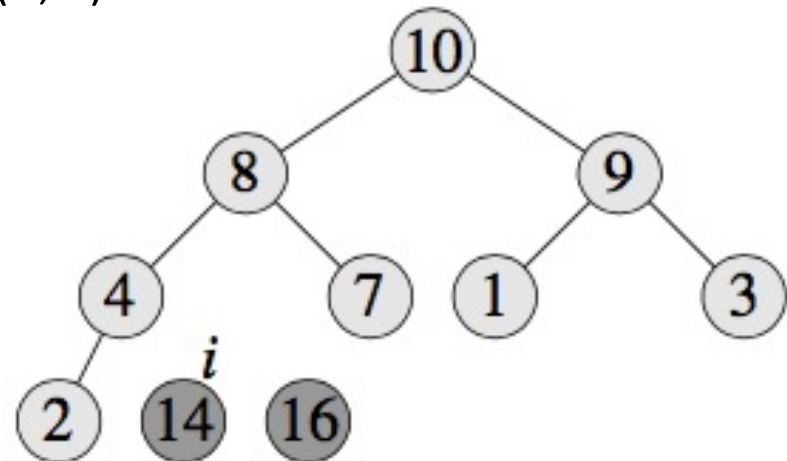
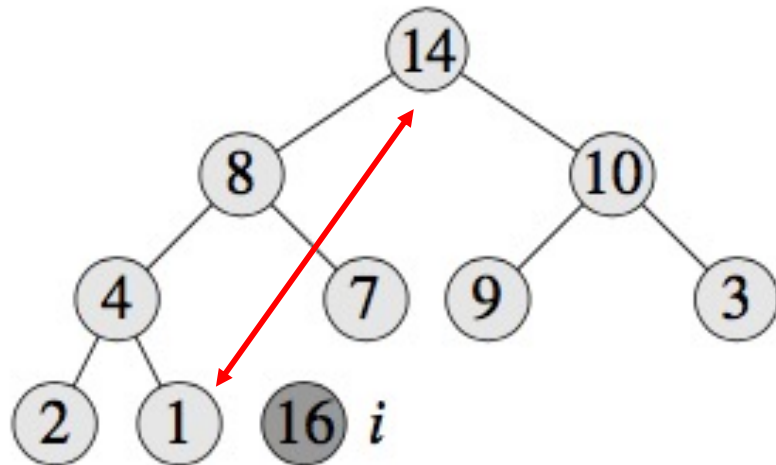
Heap Sort

`max_heapify(A, 0)`



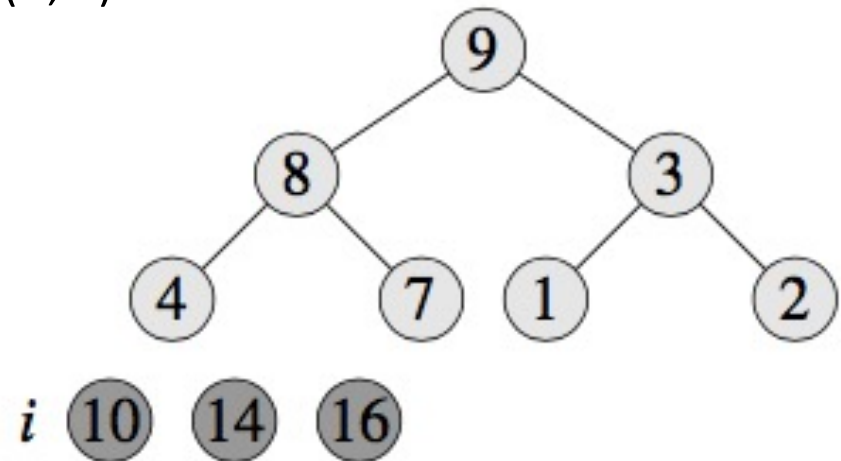
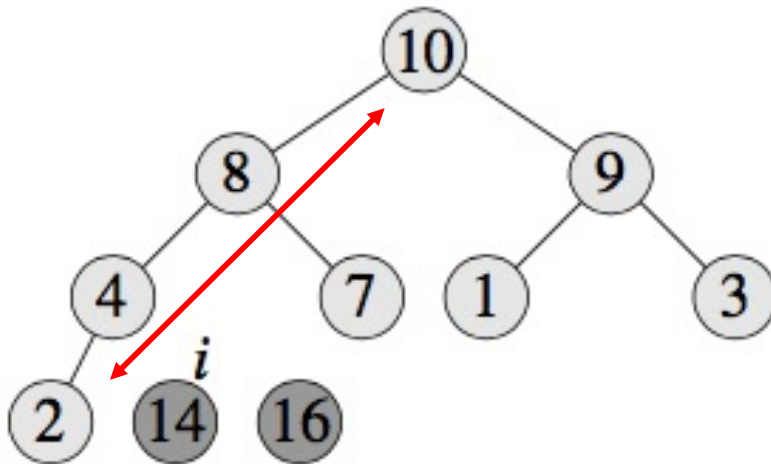
Heap Sort

`max_heapify(A, 0)`

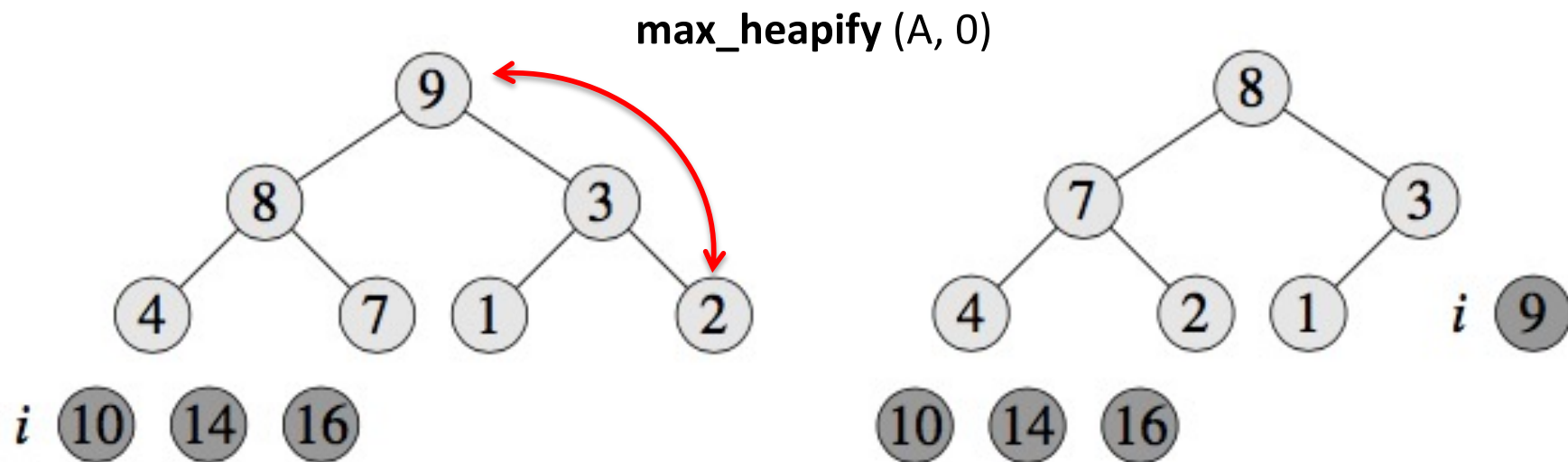


Heap Sort

`max_heapify (A, 0)`

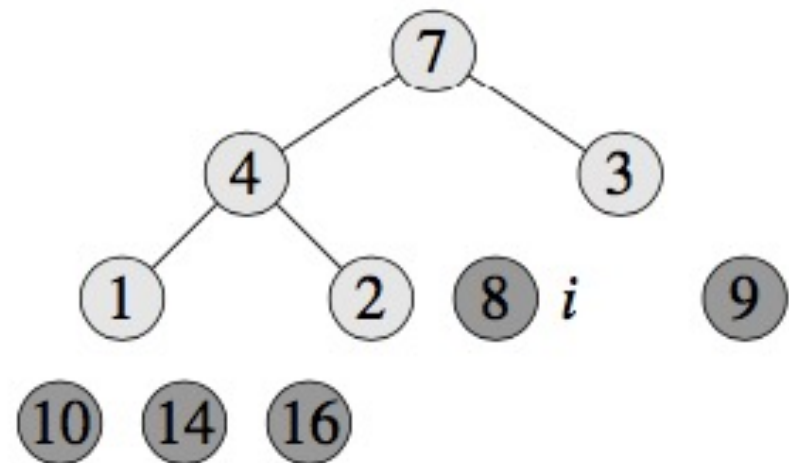
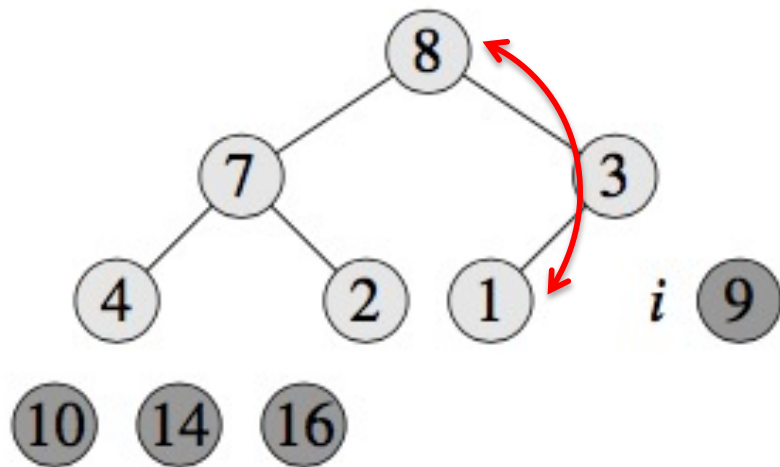


Heap Sort



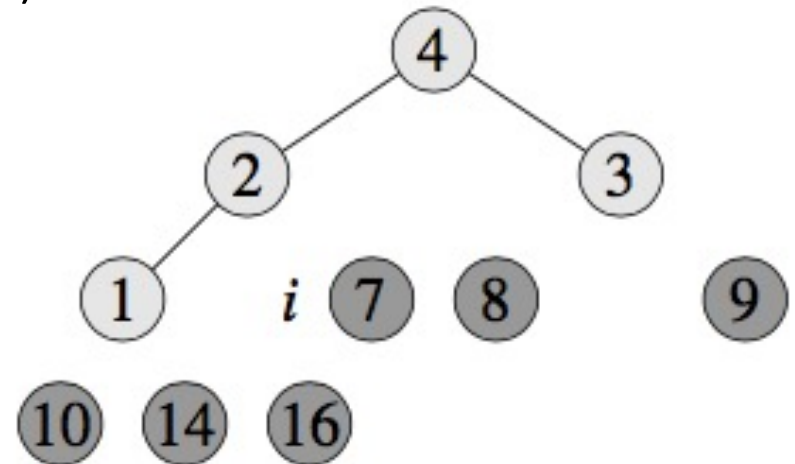
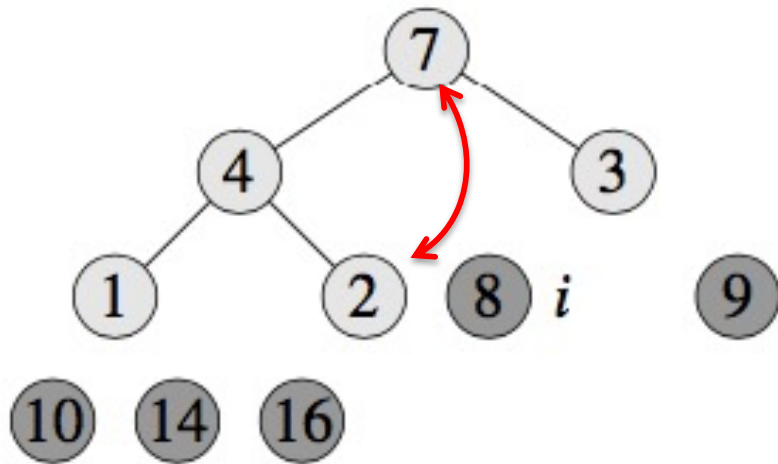
Heap Sort

`max_heapify (A, 0)`



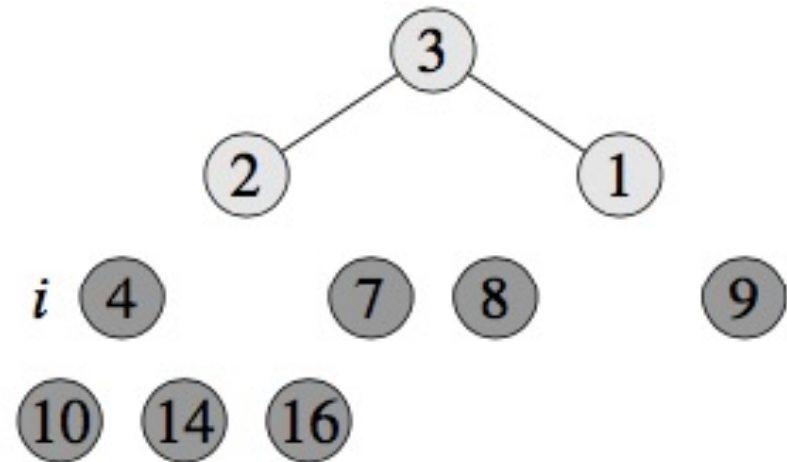
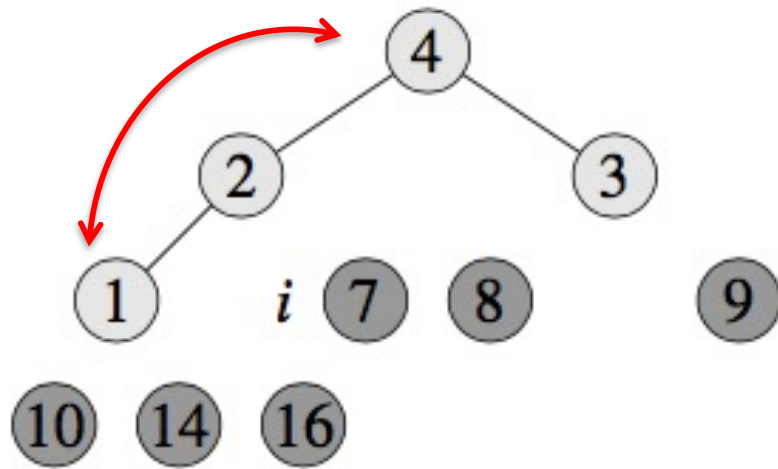
Heap Sort

`max_heapify(A, 0)`



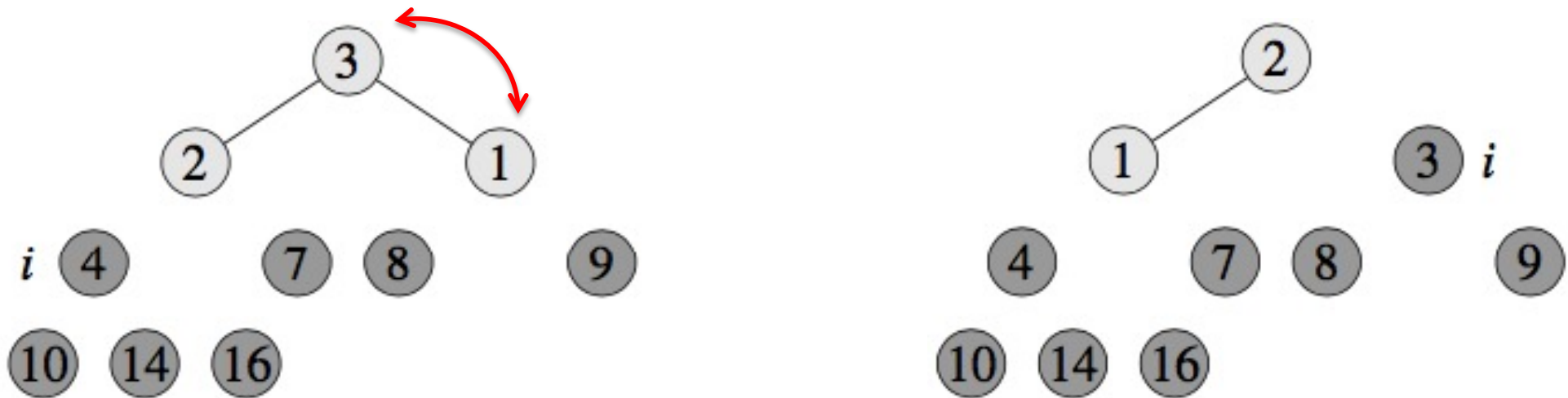
Heap Sort

`max_heapify(A, 0)`



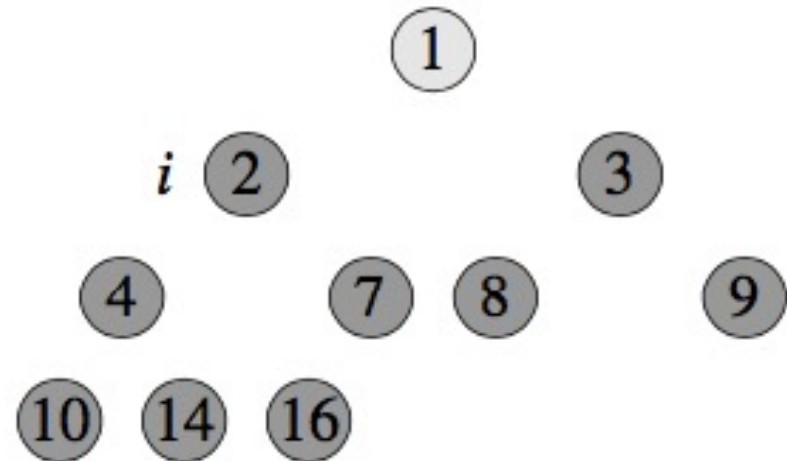
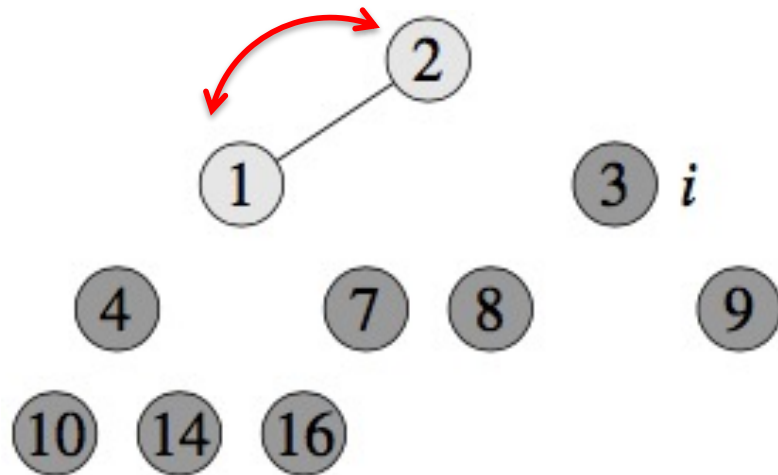
Heap Sort

`max_heapify (A, 0)`



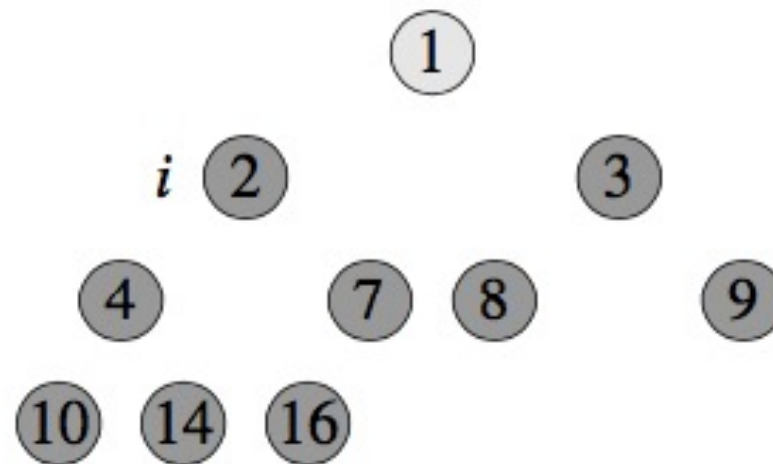
Heap Sort

`max_heapify (A, 0)`



Heap Sort

Vettore ordinato in ordine inverso e memorizzato nello stesso array che prima memorizzava l'albero.



<i>A</i>	1	2	3	4	7	8	9	10	14	16
----------	---	---	---	---	---	---	---	----	----	----

Heap Sort

Implementare in c...

Heap Sort

```
void heapsort(int* A, int heap_size) {  
    int tmp;  
  
    build_max_heap(A, heap_size);  
    printArray(A, heap_size);  
    for (int i=heap_size; i>1; i--) {  
        // swap radice con ultimo elemento  
        tmp=A[i-1];  
        A[i-1]= A[0];  
        A[0]=tmp;  
  
        max_heapify(A, 0, i-1);  
    }  
}
```

Heap Sort

Complessità computazionale

- La chiamata di BUILD-MAX-HEAP impiega $O(n)$
- Ciascuna delle $n - 1$ chiamate di MAX-HEAPIFY impiega $O(\lg n)$.

HEAPSORT impiega quindi $O(n \lg n)$.

Complessità computazionale

Caso Peggior:

insertion sort: $O(n^2)$

merge sort: $O(n \lg n)$

heapsort: $O(n \lg n)$.

NB: i fattori costanti di insertion sort lo rendono piu` veloce per piccoli valori di n .

Heap Sort

La sequenza $\langle 23, 17, 14, 6, 13, 10, 1, 5, 7, 12 \rangle$ è un max-heap?

Illustrate l'azione di BUILD-MAX-HEAP sull'array $A = \langle 5, 3, 17, 10, 84, 19, 6, 22, 9 \rangle$. Utilizzate le figure dell'albero analogamente a quanto fatto sulle slides.

Perché l'indice i del ciclo di BUILD-MAX-HEAP deve diminuire da $\lfloor \text{lunghezza}[A]/2 \rfloor$ a 1, anziché aumentare da 1 a $\lfloor \text{lunghezza}[A]/2 \rfloor$?

Heap Sort

Illustrate l'azione di HEAPSORT sull'array $A = \langle 5, 13, 2, 25, 7, 17, 20, 8, 4 \rangle$. Utilizzate le figure dell'albero analogamente a quanto fatto sulle slides.

Qual è il tempo di esecuzione di heapsort con un array A di lunghezza n che è già ordinato in senso crescente? E se l'array è ordinato in senso decrescente?

CODE DI PRIORITÀ

Applicazioni piu` diffuse dell'heap: coda di priorità.
(code di max-priorità e code di min-priorità).

Una coda di priorità è una struttura dati che serve a mantenere un insieme S di elementi, ciascuno con un valore associato detto chiave.

Una coda di max-priorità supporta le seguenti operazioni.

CODE DI PRIORITÀ

Un'applicazione delle code di max-priorità è quella di schedulare i task.

La coda di max-priorità tiene traccia dei compiti da svolgere e delle loro relative priorità. Quando un compito è ultimato o interrotto, viene selezionato il compito con priorità più alta fra quelli in attesa mediante EXTRACT-MAX. Un nuovo compito può essere aggiunto alla coda in qualsiasi istante mediante INSERT.

CODE DI PRIORITÀ

MAXIMUM(S) restituisce l'elemento di S con il valore piu` grande.

INSERT(S,x) inserisce l'elemento x nell'insieme S. Questa operazione potrebbe essere scritta cos`i: $S \leftarrow S \cup \{x\}$.

EXTRACT-MAX(S) elimina e restituisce l'elemento di S con il valore piu` grande.

INCREASE-KEY (S, i, x) aumenta il valore dell'elemento i al nuovo valore x, che si suppone sia almeno pari al valore corrente dell'elemento i.

DECREASE-KEY idem.

Le funzioni devono accettare un max-heap e rendere un max-heap

Implementare le funzioni per la struttura duale min-heap

CODE DI PRIORITÀ

MAXIMUM(S) restituisce l'elemento di S con il valore piu` grande.

INSERT(S,x) inserisce l'elemento x nell'insieme S. Questa operazione potrebbe essere scritta cos`i: $S \leftarrow S \cup \{x\}$.

EXTRACT-MAX(S) elimina e restituisce l'elemento di S con il valore piu` grande.

INCREASE-KEY (S, i, x) aumenta il valore dell'elemento i al nuovo valore x, che si suppone sia almeno pari al valore corrente dell'elemento i.

DECREASE-KEY idem.

Le funzioni devono accettare un max-heap e rendere un max-heap

Implementare le funzioni per la struttura duale min-heap

L'implementazione è lasciata come esercizio.
Nell'implementazione fate riferimento alle funzioni per la manipolazione dell'heap

Code priorità

Note su **increase-key** e **decrease-key**.

L'ipotesi di partenza è avere un max-heap

Dopo l'applicazione di **decrease-key** al nodo i , l'albero è nella condizione di applicabilità della max-heapify (è un max-heap tranne al più un nodo più piccolo)

Dopo l'applicazione di **increase-key** al nodo i , l'albero **NON** è nella condizione di applicabilità della max-heapify (non solo potrebbe essere $\text{parent}(i) < A[i]$, ma anche $\text{parent}(\text{parent}(i)) < A[i]$.)

Applicare max-heapify non risolve: occorre spostare il nodo VERSO L'ALTO.

Esercizio

- Fusione di due Max-Heap statici
- Specifica:

```
int mergeHeaps(int h1[], int n1, int h2[], int n2, int out[])
```

-
- Verifica se un array è già un Max-Heap
 - trovare i K elementi più grandi

FONTI

Introduzione agli algoritmi e strutture dati

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest,
Clifford Stein